

Development of a Flight Performance Assessment Service for Small Multirotor Aircraft

Justin J. Matt*, George V. Altamirano†, and Steven D. Young‡
NASA Langley Research Center, Hampton, Virginia 23681

James R. Barnes§ and Tobin Zheng¶
KBR, Hampton, Virginia 23681

This paper presents the development of a Flight Performance Assessment Service (FPAS), intended to support pre-flight and in-flight trajectory planning. The service provides users with a capability to simulate flight trajectories and predict performance characteristics for small multirotor aircraft in nominal and motor failure scenarios. Users input flight plans to the service that can be designed using conventional ground control station software along with vehicle characteristics such as the number of motors, airframe dimensions, and aircraft mass properties. Wind information may also be provided if available. FPAS outputs simulated aircraft states and performance characteristics such as battery consumption profiles, control margin, and trajectory tracking accuracy. FPAS provides high-fidelity results by utilizing a large database of aerodynamic and power consumption models developed from extensive wind tunnel testing of representative small multirotor aircraft. These aerodynamic models are combined with a guidance and control architecture that mimics typical sUAS autopilot modes and behaviors to enable waypoint following. To enable fast repeatable simulation, models were converted to C++ code using Mathworks® Embedded Coder® and integrated into a standalone software application. The developed software includes an interface to communicate with an external commercial weather service to provide wind data and atmospheric conditions to be used within the vehicle simulations. The service can be run from a command line environment or from a graphic user interface, which also provides visualizations of the simulated flight trajectories. An example is provided that compares FPAS outputs to measured flight data of a small multirotor test aircraft. The predicted outputs, which include ground trajectory and energy usage, match the flight data within acceptable margins for performance assessment and trajectory planning applications. In terms of service response times, FPAS has been demonstrated to produce the trajectory and performance predictions at approximately 100 times faster than real-time on a typical business laptop, even for complex flight plans that span long distances or contain many waypoints.

Nomenclature

$C_{x,y,z}$	=	airframe force coefficients
$C_{l,m,n}$	=	airframe moment coefficients
C_L, C_D	=	airframe lift and drag coefficients
$C_{T,H}$	=	rotor thrust and H-force coefficients
C_{M_x}, C_{M_y}, C_Q	=	rotor hub moment and torque coefficients
F	=	vehicle forces expressed in body frame, lb
M	=	vehicle moments expressed in body frame, ft-lb
P_s	=	motor shaft power, ft-lb/s
R	=	rotor radius, ft

*Research Aerospace Engineer, Flight Dynamics Branch, justin.j.matt@nasa.gov.

†Research Aerospace Engineer, Flight Dynamics Branch.

‡Research Aerospace Engineer, Safety-Critical Avionics Systems Branch.

§Senior Systems Engineer, Simulation Development and Analysis Branch.

¶Associate Technical Professional, Simulation Development and Analysis Branch.

V	=	velocity, ft/s
α, β	=	angle of attack and angle of sideslip, rad or deg
μ_x, μ_z	=	rotor advance ratio and normal velocity ratio
τ_s	=	motor shaft torque, ft-lb
ϕ, θ, ψ	=	roll, pitch, and yaw angles, deg
Ω	=	motor rotational speed, rad/s

I. Introduction

Advances in aviation technology are quickly broadening mission profiles, operational environments, the types of aerospace vehicles, and applications of automation. As a result, more flights will occur at lower altitudes, particularly in and around densely populated areas. To ensure safety in this era of expansion, new approaches to safety assurance will need to be added to those already in place for traditional aviation (e.g., today's commercial airline operations). With this challenge in mind, NASA and the National Academies have suggested a set of research and development goals and a concept called In-Time Aviation Safety Management Systems (IASMS) [1, 2]. This concept builds on and complements existing SMS pillars and standards [3], with a goal of not only providing more timely mitigation of risk during operations, but also better coverage of latent (sometimes unknown) risks. This is accomplished via a set of IASMS capabilities tailored to (1) mission type, (2) vehicle type (and its equipage), (3) operational environment, and (4) safety risk tolerance for the operation [4]. To be most effective, a tailored IASMS should be designed to an appropriate capability coverage that spans:

- Pre-flight, in-flight, and post-flight capabilities
- Monitoring, assessment, and mitigation capabilities
- Reactive, proactive, and predictive capabilities

In order to refine an initial Concept of Operations (ConOps), explore various Concepts of Use (ConUses), and expose/validate requirements for future IASMS, testing architectures were created. Then, a set of capabilities and an underlying information system was conceived and developed based on hazards associated with two domains: (1) highly autonomous small uncrewed aerial systems (sUAS) operating at low altitudes within and over urban areas and (2) highly autonomous air taxis operating within and over urban areas. The initial scope of this development is described in [5], with updates provided in [4]. This paper discusses one of the capabilities developed for domain (1); however, it may be generally applicable to domain (2) as well.

As stated in [4, 5], it is important to note that there are many capabilities already in use by the industry (or soon to be in use) that will play critical roles in future IASMS designs. The ones investigated and advanced by NASA and its partners address gaps in the current state-of-the-art regarding specific hazards and/or allow for investigation of the interplay between and across hazard types — particularly regarding how overall safety risk can be reduced or managed more effectively through proactive and predictive capabilities.

IASMS capabilities may be implemented as a combination of information services, embedded functions, and human roles. This paper outlines the development of one such service, a Flight Performance Assessment Service (FPAS) that could be used as part of a pre-flight and/or in-flight risk assessment capability. FPAS provides information based upon flight dynamics and performance models to predict and assess flight trajectories, power consumption, and operational envelope limits, especially with regards to how they may deviate from expectations or the nominal case (e.g., due to wind). FPAS utilizes a vast aerodynamic database created from the results of multiple experiments conducted in the NASA Langley Research Center (LaRC) 12-Foot Low Speed Tunnel of small multirotor aircraft and their components [6–10]. The database is then used to accurately simulate flight trajectories for these types of aircraft with respect to a user-specified flight plan. FPAS is designed using a generalized, modular framework that allows for multirotor vehicles with various characteristics to be simulated in order to provide the user-requested flight performance metrics. When used during pre-flight planning, iterative use of FPAS allows users to create and/or modify flight plans to achieve acceptable flight performance-related margins. When used in-flight, FPAS algorithms are implemented as onboard functions to support automated contingency management (e.g., informing re-route decision logic).

This paper is organized as follows. Section II introduces the top-level architecture of FPAS, including inputs, outputs, and ways to interface and interact with the service. Section III outlines the development of the underlying multirotor simulation models. The software development process is detailed in Section IV. An example comparison between FPAS predictions and flight data is shown in Section V. Finally, Section VI presents conclusions and discusses future research directions.

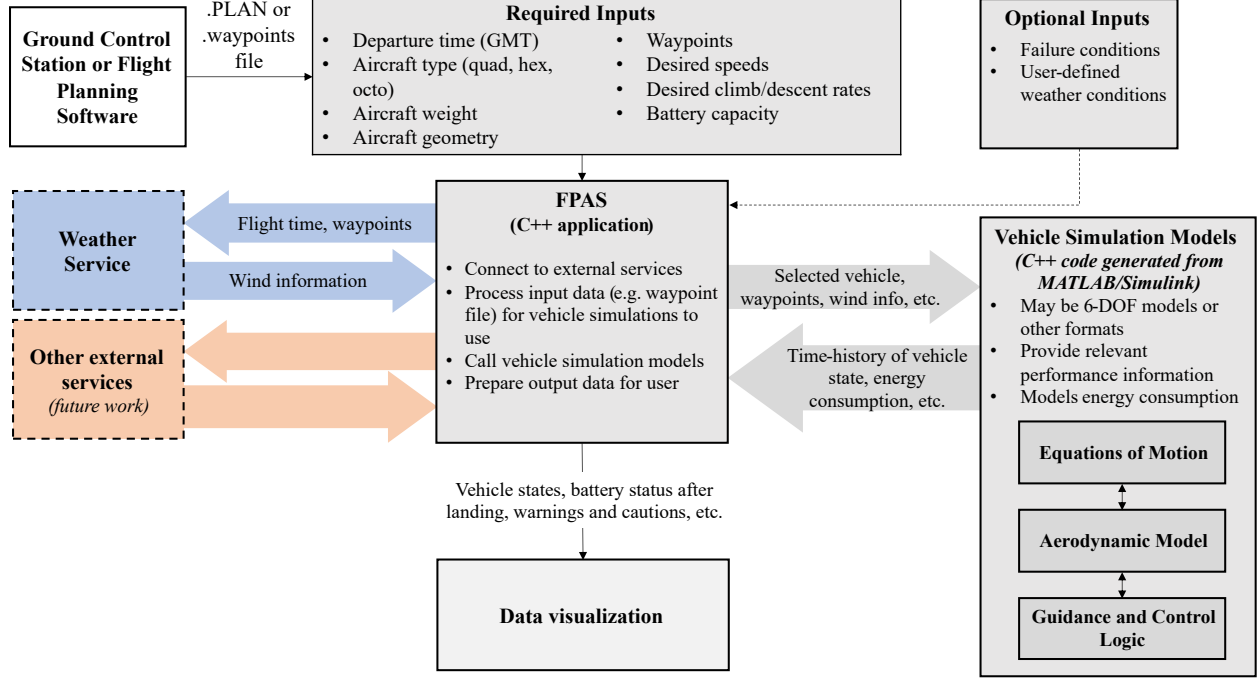


Fig. 1 FPAS Architecture Diagram.

II. FPAS Architecture

FPAS was developed to support pre-flight and in-flight trajectory planning for small multirotor aircraft. The service generates detailed trajectory and vehicle performance predictions for a user-specified aircraft and flight plan. In addition, warning messages are provided to the user to highlight portions of the flight plan where the vehicle experienced low battery level, limited control margin, or deviated from the intended trajectory beyond a specified threshold. A high-level architecture diagram of FPAS is shown in Fig. 1. The service parses and prepares user input data to be used within one of several different vehicle simulation models. The simulation models utilize empirical data sets gathered from extensive wind tunnel testing performed at NASA LaRC [6–10]. These data sets were used to create a six degree of freedom (6-DOF) simulation model that includes waypoint following guidance and control algorithms similar to those found in popular open-source autopilots such as ArduPilot or PX4. The simulation architecture is largely based on the simulations originally developed in [6]. FPAS simulation models were originally developed in Simulink® and converted to a standalone C++ application with a graphic user interface (GUI). FPAS simulates trajectories at around 100 times faster than real-time when tested for benchmarks using an Intel i5-1345U CPU, which is representative of a typical laptop for business. The relative execution speed was found to be consistent regardless of the complexity of the flight plan, and the total run-time increases linearly with the length of the simulation.

FPAS is designed to be modular, so that vehicle simulation models can easily be added to the service. Currently, users can select between different types of multirotor aircraft, including quadcopters, hexacopters, and octocopters. Additionally, the modular framework facilitates communication between FPAS and other external services to send and receive data in support of a large IASMS framework. For example, an external weather service can be queried to provide wind and atmospheric information to the vehicle simulation models. This section will outline the top-level architecture of FPAS.

A. User Inputs

Inputs to FPAS include a user-generated flight plan and several manually specified inputs. Two file types are supported for flight plans based on the files generated by the ground control station (GCS) software Mission Planner (.waypoints files) and QGroundControl (.PLAN files). These programs are used to define a flight plan that is supported by the MAVLink protocol, which is a commonly used protocol for sUAS communication. Flight plans consist of a series of mission commands, such as taking off or flying to a waypoint defined by longitude, latitude, and altitude. Mission commands also include parameters that modify the definitions. For example, waypoint commands include a parameter

defining the acceptance radius associated with the waypoint. These flight plans also embed other information, such as velocity commands and geofences, which are supported by FPAS. FPAS reads the uploaded flight plan and creates a universal comma-separated values (.csv) file that is used by the vehicle simulation models.

In addition to the flight plan, FPAS users must manually specify other information before running the service. First, the user must select the vehicle model used to simulate the flight plan. Currently, generic 6-DOF models for a quadcopter, hexacopter, and octocopter are included within FPAS. These models are generalized so that the user can specify characteristics of the vehicle they want to simulate, such as weight, motor position, rotor size, and airframe size. Users also specify the default velocity and climb rates that the aircraft will maintain if not otherwise specified in the flight plan and the energy capacity of the aircraft battery. Additionally, users can specify wind and temperature information or choose to look up the information from a commercial weather service based on the coordinates of the flight plan [11]. Finally, optional inputs include specifications to simulate a motor failure during flight. The motor failures are characterized as either a complete loss of torque or a stuck throttle command that can be applied to one or more of the motors. The user can specify the time at which the failure is injected into the simulation.

B. FPAS Outputs

After running FPAS, five files are created:

- The flight plan .csv file is saved.
- A second .csv file is saved with vehicle simulation time series data.
- A plain-text log file is created that provides information, warnings, and cautions about the simulated flight plan.
- If the flight plan includes an embedded geofence, a .JSON file defining the geofence is saved.
- A telemetry log file (.tlog) is artificially created so that the simulation data can be played back in GCS software.

The vehicle simulation data is dependent on the specific vehicle model simulated. Typical outputs include the simulated vehicle position, attitude, velocity, motor speed, motor power, total energy consumption, and battery level, which is computed simply by subtracting the simulated total energy consumption from the user-specified battery capacity. To create the log file, FPAS computes several variables from the simulation data after it has finished running. This includes the remaining battery percentage, the control margin of the motors, the deviation from the desired trajectory, and position of the vehicle relative to the geofence. If the value of any of these variables exceeds an internally defined threshold, a new message will be created in the log file with a timestamp and caution or warning message.

C. Interfacing

Interfacing with FPAS can be achieved through a command line or scripting interface or using the developed GUI. The command line or scripting environment provides a convenient way to run and analyze multiple cases quickly, while the GUI provides a more interactive experience and allows for quick and simple visualization of the simulation data. FPAS runs as a single executable file that reads from an initialization file in the same directory. The initialization file is a plain-text file that includes definitions for all the required user inputs and a path to the flight plan file. The GUI provides a dialogue to modify the initialization file, runs the executable from a button press, and provides visualization of the simulation output data and log message data. An example of the GUI input dialogue and visualization are shown in Figs. 2 and 3.

The GUI visualization includes five main components. In the top left, the ground track of the simulated flight is plotted on top of the uploaded flight plan. Any defined geofences are included in this plot, as shown in Fig. 3. The top right section of the GUI includes a data plotter, where users can create two-dimensional plots from any of the simulation data. In the example, airspeed and groundspeed are plotted against time. In the bottom left, the altitude is plotted against time. In the bottom center, the alert messages from the log file are shown. In the bottom right, flight status and weather information are shown, including battery status and a warning indicator. On the bottom of the GUI is a scrollbar that allows users to scroll through the simulated flight time or play back the flight.

Initialization Form
?
X

Max Simulation Time:

6000.0

s

▼

Recording Rate:

0.1

s

▼

Departure Time:

7/14/2023 12:00 PM

▼

Vehicle Type:

GenericOctocopter

▼

Vehicle Weight:

15

lb

▼

Waypoint File:

flight_plan.plan

Select File

Desired Speed:

15

knot

▼

Desired Speed Type:

GROUND_SPEED

▼

Desired Climb Rate:

500

ft/minute

▼

Desired Descent Rate:

500

ft/minute

▼

Wind Speed:

3

knot

▼

Wind Direction:

55

deg

▼

Temperature:

60

degF

▼

Battery Capacity:

500

Wh

▼

Battery Charge (%):

100

R Prop:

0.583

ft

▼

D Airframe:

3.44

ft

▼

Automatic Motor Positions:

TRUE

▼

Motor Failures:

[0 0 0 0 0 0 0]

Motor Failure Magnitude:

[0 0 0 0 0 0 0]

Motor Failure Time:

0

s

▼

Weather Service Type:

none

OK

Cancel

Fig. 2 FPAS GUI initialization dialogue.

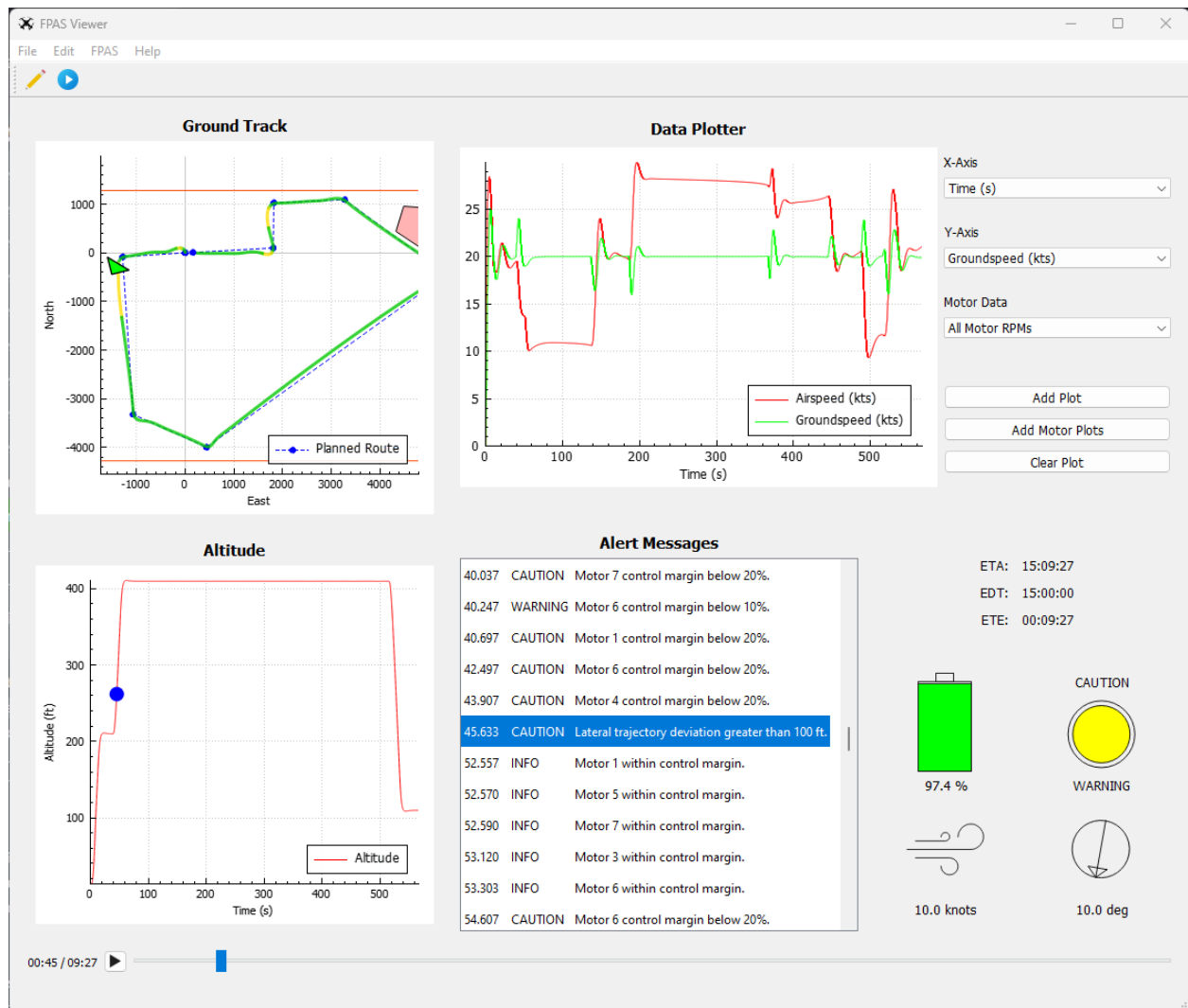


Fig. 3 FPAS GUI output visualization.

III. Multirotor Simulation Models

The simulation model was developed to represent any generic, small multirotor aircraft. Databases from wind tunnel tests of rotors, quadcopters, and octocopters are used to compute the aerodynamic forces and moments within the simulation [6, 7, 9, 10].

A. Equations of Motion

The simulation is based on 6-DOF rigid-body dynamics. The dynamics can be expressed as 12 coupled, nonlinear, differential equations [12]:

$$\dot{\Phi} = H(\Phi)\omega_{b/e}^b \quad (1a)$$

$$\dot{p}_{cm/e}^e = C_{e/b}\mathbf{v}_{cm/e}^b \quad (1b)$$

$$\dot{\mathbf{v}}_{cm/e}^b = \frac{1}{m}\mathbf{F}^b + C_{b/e}\mathbf{g}^e - \tilde{\omega}_{b/e}^b\mathbf{v}_{cm/e}^b \quad (1c)$$

$$\dot{\omega}_{b/e}^b = (\mathbf{I}^b)^{-1} [\mathbf{M}^b - \tilde{\omega}_{b/e}^b\mathbf{I}^b\omega_{b/e}^b] \quad (1d)$$

Equation (1a) describes the rotational kinematics, which relate the vehicle attitude angles, Φ , to the angular rates expressed in the body frame, $\omega_{b/e}^b$. The translational kinematics are described in Eq. (1b). The body velocities, $\mathbf{v}_{cm/e}^b$, are related to the derivative of the vehicle center of mass (CM) position expressed in an earth-fixed North-East-Down (NED) coordinate frame by an orthogonal rotation matrix $C_{e/b}$. Equations (1c)-(1d) describe the translational and rotational dynamics, which are functions of the body frame forces and moments, $\mathbf{F}^b$ and $\mathbf{M}^b$, the gravity vector, $\mathbf{g}^e$, the vehicle mass, m , and inertia matrix, $\mathbf{I}^b$, and the skew-symmetric matrix cross product matrix $\tilde{\omega}_{b/e}^b$, which is computed from the vector $\omega_{b/e}^b$.

In addition to the rigid body equations of motion, the motor dynamics were also modeled. The motor rotational speeds, Ω_m , were modeled using linear first-order differential equations,

$$\dot{\Omega}_m = \frac{K_m}{\tau_m}\delta_m - \frac{1}{\tau_m}\Omega_m \quad (2)$$

where δ_m is the throttle command for the motor ranging from 0 to 100, τ_m is the motor time constant, and K_m is a gain that relates the throttle signal to the steady state motor speed in rad/s. Values for the parameters τ_m and K_m were estimated from empirical data collected from bench testing [10]. A time constant of 0.12 seconds was identified and found to be representative of different motor and rotor sizes. The motor speed gain is modeled as a function of the rotor radius, R :

$$K_m = 2\pi \frac{R_{\text{ref}}}{R} \quad (3)$$

where $R_{\text{ref}} = 8$ in is the reference radius of the primary rotor that was tested to identify the parameters. Equations (2) and (3) are very simplified models of electric motor dynamics. The benefit of this approach is in its simplicity – a variety of motor and rotor sizes can be simulated accurately enough for typical FPAS use-cases (e.g., nominal operation) without any user knowledge about the motors or electronic speed controllers onboard the vehicle. However, a drawback compared to a physics-based modeling approach is that the simulated motor response can be less realistic, particularly outside of the region of linear dynamics. For example, different types of motor failure scenarios may be implemented in a more straightforward manner using physics-based models.

B. Forces and Moments

The aerodynamics are modeled used a build-up approach, where forces and moments from individual rotors and the airframe are modeled separately and superimposed to compute the total vehicle forces and moments.

$$\mathbf{F} = \mathbf{F}_A + \mathbf{F}_P \quad (4a)$$

$$\mathbf{M} = \mathbf{M}_A + \mathbf{M}_P \quad (4b)$$

1. Rotor Aerodynamics

The rotor aerodynamics are modeled using a look-up table that was developed from extensive wind tunnel testing of an isolated rotor [6]. The look-up table outputs the six rotor force and moment coefficients as a function of the rotor advance ratio and normal velocity ratio

$$\mu_x = \frac{V \cos(i_p)}{\Omega R} \quad (5a)$$

$$\mu_z = \frac{V \sin(i_p)}{\Omega R} \quad (5b)$$

where i_p is the rotor incidence angle relative to the flow. The rotor force and moment coefficients are defined as

$$C_T = \frac{T}{\rho \pi R^2 (\Omega R)^2} \quad C_H = \frac{H}{\rho \pi R^2 (\Omega R)^2} \quad (6a)$$

$$C_{M_x} = \frac{M_x}{\rho \pi R^3 (\Omega R)^2} \quad C_{M_y} = \frac{M_y}{\rho \pi R^3 (\Omega R)^2} \quad C_Q = \frac{Q}{\rho \pi R^3 (\Omega R)^2} \quad (6b)$$

The rotor forces are the thrust, T , and H-force, H , and the moments are the rotor hub rolling and pitching moments, M_x and M_y , and the aerodynamic torque, Q . The thrust and torque coefficient models are shown in Fig. 4.

The motor shaft torque is computed from the aerodynamic torque, rotational acceleration, and moment of inertia of the rotating motor-rotor assembly about its axis of rotation, I_z .

$$\tau_s = \dot{\Omega} I_z + Q \quad (7)$$

It is worth noting that typical physics-based models would compute acceleration from the shaft torque, whereas Eq.(7) effectively "back-computes" the torque from the acceleration. The shaft power is then computed from the product of the motor torque and motor speed.

$$P_s = \Omega \tau_s \quad (8)$$

Electrical power consumption is computed by dividing the mechanical power by an efficiency factor of 90%, which was selected based on values identified from flight testing for system identification of an octocopter in a hover [13]. Finally, the total vehicle energy consumption is computed by integrating the total electrical power consumption over time.

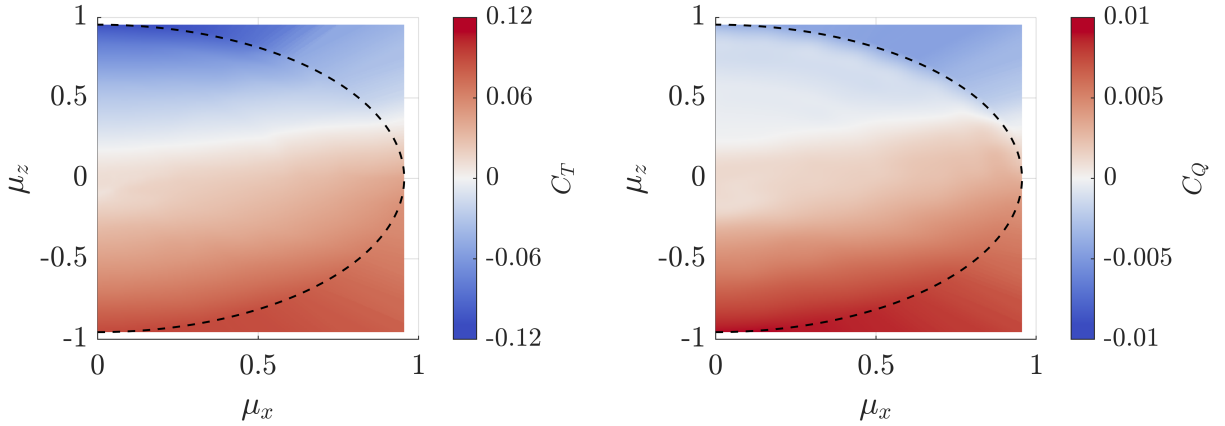


Fig. 4 Rotor thrust and torque coefficient look-up table models. Areas outside the half circles are linearly extrapolated from experimental data.

2. Airframe Aerodynamics

Force and moment coefficients for the isolated airframe are similarly derived from wind tunnel data collected in [9]. The force and moment coefficients in the body frame are functions of the angle of attack, α , and angle of sideslip, β , and are defined as:

$$C_{x,y,z} = \frac{F_{x,y,z}}{0.5\rho V^2 S} \quad (9a)$$

$$C_{l,m,n} = \frac{M_{x,y,z}}{0.5\rho V^2 S l} \quad (9b)$$

where ρ is the air density, V is airspeed, S is a reference area defined as the circular area that encompasses the airframe from a top-down view, and l is reference length equal to the radius of the circular area used to define S . The look-up tables span from -90 to 90 deg for both angle of attack and sideslip. An example of the lift, drag, and pitching moment coefficient are shown in Fig. 5. In order to model flight conditions outside this range, such as rearward flight, the vehicle aerodynamics are assumed to be symmetric. Note that the lift and drag coefficients, C_L and C_D , are computed from rotating the body X- and Z- direction force coefficients about the angle of attack.

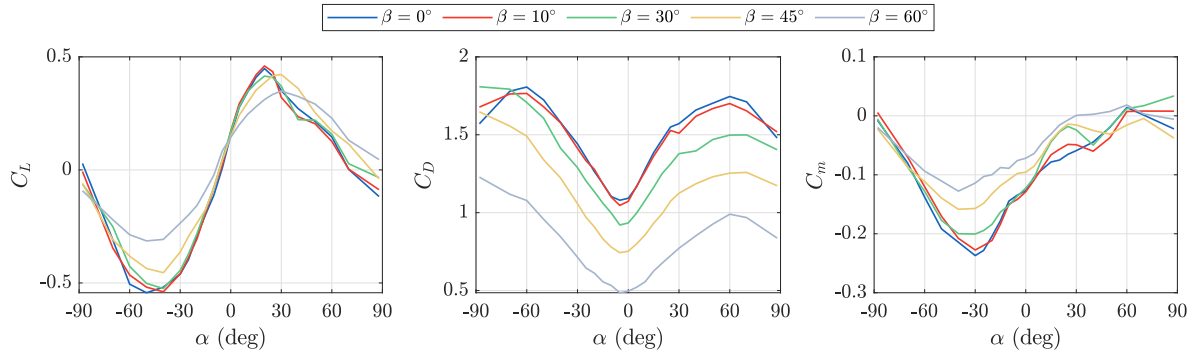


Fig. 5 Airframe lift, drag, and pitching moment coefficients.

C. Guidance and Control Algorithms

The simulation includes guidance and control algorithms that are designed to follow a flight plan defined by discrete waypoints. Waypoint definitions include the desired latitude, longitude, and altitude for the vehicle to fly towards and the desired velocity for the vehicle to maintain. The algorithms accommodate both groundspeed- and airspeed-based guidance, which can be selected by the user. A cascaded guidance and control architecture is used, as shown in Fig. 6. The guidance and control algorithms were originally tuned using a single linearized model of an octocopter and then refined to ensure that performance was adequate for various vehicle configurations and user inputs.

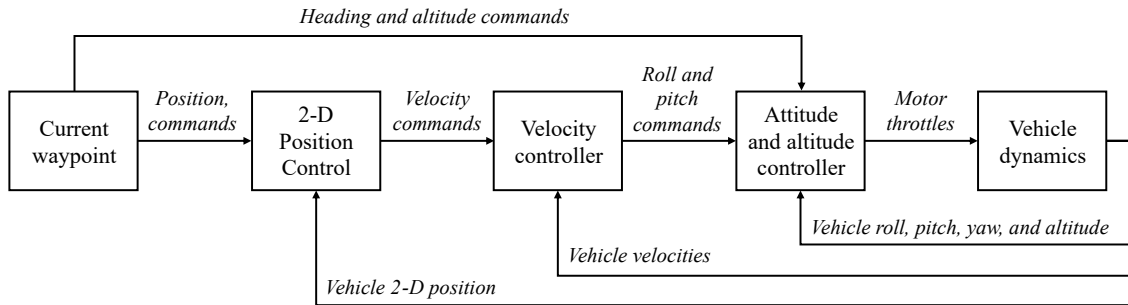


Fig. 6 Guidance and control architecture.

IV. Software Development

This section outlines the development of FPAS as a standalone executable application. Software development began with the generation of C++ code from the Simulink models through the use of Mathworks® Embedded Coder®. The generated code was then incorporated into the Langley Realtime Simulation in C++ (LaSRS++) environment, which is an object-oriented framework developed at NASA LaRC that has been used extensively to support real-time simulation of aircraft [14]. A graphic user interface was then developed to interact with FPAS and analyze output data.

A. Simulation

The FPAS simulation models were converted to C++ using Embedded Coder® on the root level of the Simulink model. Two scripts were run to ensure that the models were properly configured for real-time operations. The first script locates continuous Simulink blocks in the Simulink model. Any blocks that are located must be replaced with discrete-time versions to work with the external simulation. The next script configures the code generation parameters appropriate to the model and real-time operations, such as code generation fields, setup optimization fields, and parameters to turn off dynamic memory allocation.

The generated C++ classes are then added to the FPAS build environment as a separate static library for the vehicle. Figure 7 shows a Unified Modeling Language (UML) class diagram that provides a visual representation of the object oriented design of the simulation and how its classes relate to one another. New classes are colored green and existing classes are colored yellow. As shown, five new classes are required to implement a new vehicle into the simulation code. The GenericHexrotor class is the main interface for the vehicle and the other simulation classes. This class contains the GenericHexrotorSimulinkInterface, which provides a wrapper to the generated Simulink code. The recording set records specific data from the Simulink model that is not stored elsewhere in the simulation, and the registration class provides a static registration of the vehicle builder. This allows the vehicle type to be requested and built on demand by the user without making the rest of the simulation dependent on the new vehicle.

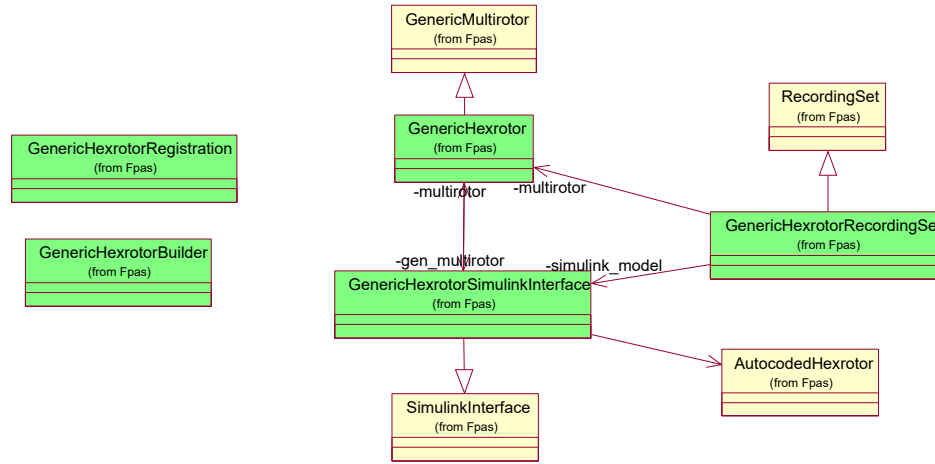


Fig. 7 FPAS classes for the GenericHexrotor vehicle.

The FlightPerformancePredictor class, shown in Fig. 8, contains the update loop of the simulation. The loop will update the vehicle, recording system data and telemetry data. This loop will continue until the Simulink model reports that the run is complete or when the vehicle altitude is less than zero. At the end of the loop, the data recording system will write out its data to a file.

The vehicle model was designed to be integrated with LaSRS++. Different data models are used to store Simulink-computed outputs and calculations for inputs to the Simulink model, which are shown in Fig. 9. Each data model has a paired recording set, as seen in the example in Fig. 10. When the recording set is updated, it queries its data class and stores the requested data to memory for future recording to file. When writing the data to file, the recording set provides a header for the data type and then formats the data in comma separated format. The user can specify the recording rate separately from the simulation rate, which runs at 300 Hz.

The simulation vehicle obtains its atmospheric properties from the World class. The atmosphere can be set to use user-provided wind and temperature data or use an external weather service. Currently, FPAS has implemented an

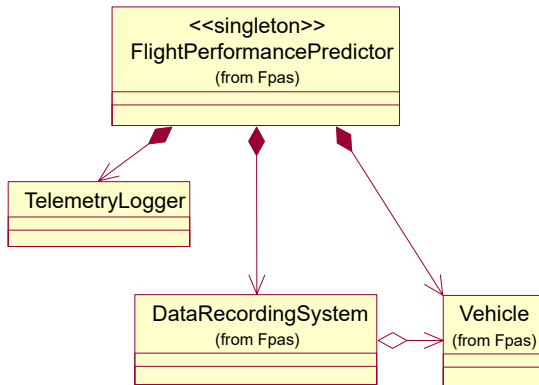


Fig. 8 FlightPerformancePredictor class.

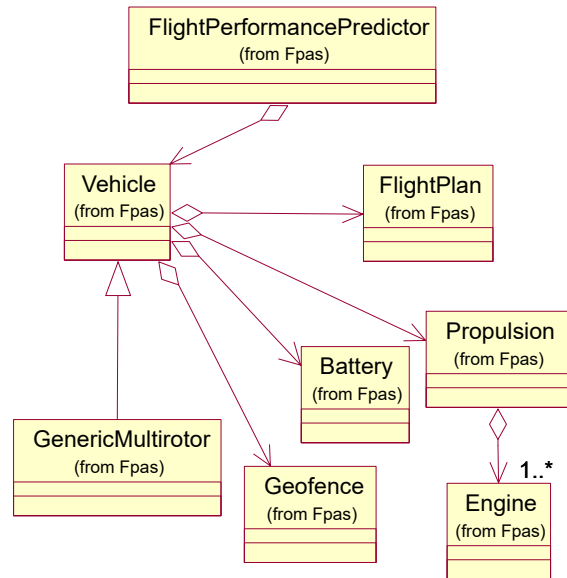


Fig. 9 Data models for storing simulation model outputs.

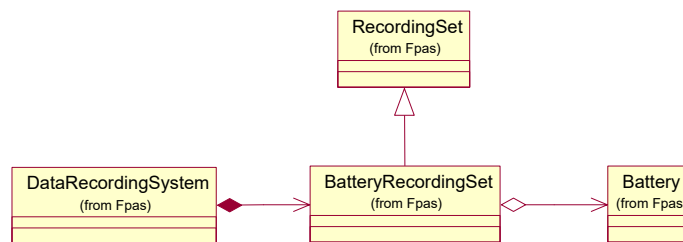


Fig. 10 Example recording set for the Battery data model.

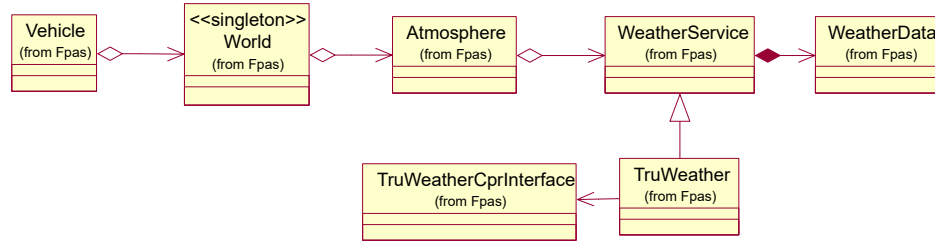


Fig. 11 Software architecture for simulation atmospheric properties.

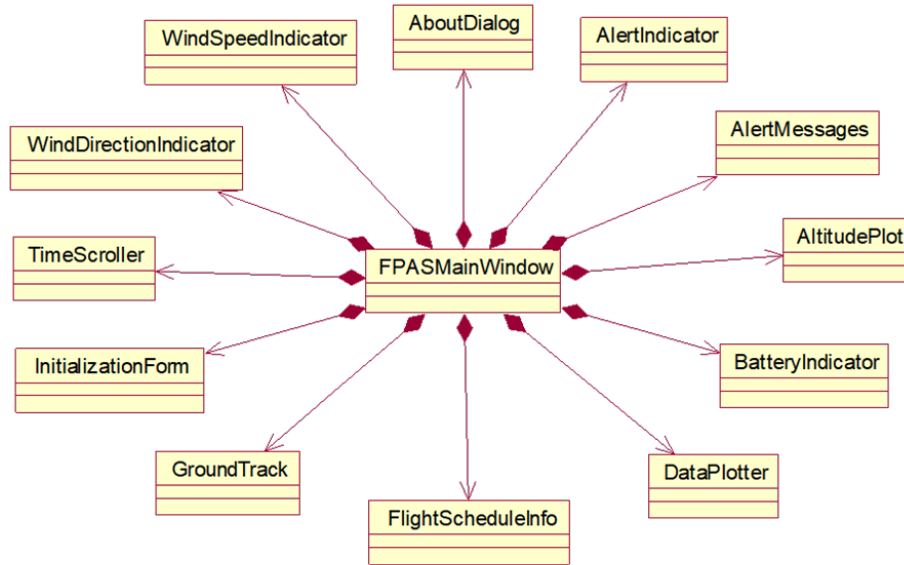


Fig. 12 GUI class interfacing.

interface to TruWeather weather service. The API to this service allows FPAS to request wind and weather information at a specified position (which are selected based on the flight plan) and time, up to one week in the future. Figure 11 outlines this architecture. When the vehicle is initialized, FPAS will download the current weather for each waypoint in the flight plan and store the data into look-up tables. To download the weather, the TruWeatherCprInterface class makes web requests to the TruWeather service using the *cpr* library. The TruWeather class then processes the downloaded weather data into a format accepted by the WeatherData class, which creates look-up tables using processed data. Whenever FPAS requests the weather at the vehicle's current location, WeatherData performs a table look-up.

B. GUI

The GUI was developed in C++ with dependencies on the Qt 5.15.2 Framework and the QCustomPlot 2.1.1 library. It features cross-platform compatibility with Windows and Linux. Each class inherits from the Qt Framework and represents a component of the GUI. Each component implements frontend and backend functionality. All classes are managed by the FPASMainWindow class, shown in Fig. 12. The class implements the GUI's main interface, establishes communication between components, parses FPAS output files, and enables users to run the FPAS simulation within the GUI. FPAS requires two arguments to perform a run: the disk location of an initialization file and the disk location of a directory for output files after a run. Both are provided by the GUI. When FPAS finishes a run, a status message on the GUI notifies the user whether the run was successful. FPASMainWindow then resets existing component models and automatically parses and loads run outputs within the output directory.

As discussed, the initialization file can be modified through a dialogue on the GUI. This dialogue is implemented

by the `InitializationForm` class. The class reads an initialization file when the GUI is launched and populates its model with the parsed content. When the user makes modifications to entries within the dialogue and presses the “OK” button, data within the model of the `InitializationForm` is saved to the initialization file. The content of the initialization file remains unchanged for non-modified entries.

The horizontal scrollbar on the bottom of the GUI allows the user to seek through the data of an FPAS run over time. As the user seeks through the data, each component connected to the scrollbar will update its visualization of the data based on the position of the scrollbar thumb. A play/pause button provides the user the ability to playback a run in real time and pause at a specific point. The graphics and capabilities of the horizontal scrollbar and the play/pause button are implemented by the `TimeScroller` class. In the backend, the class maps each scrollbar thumb position to a frame in the run data. To allow playback, the class updates the thumb position based on a timer which keeps track of real time passed and the run frame rate. Each of the main components: ground track, data plotter, altitude plot, alert messages, flight information and indicators, are connected to the `TimeScroller` class. The alert messages and ground track components additionally allow the user to seek to a point where an alert occurs in the run.

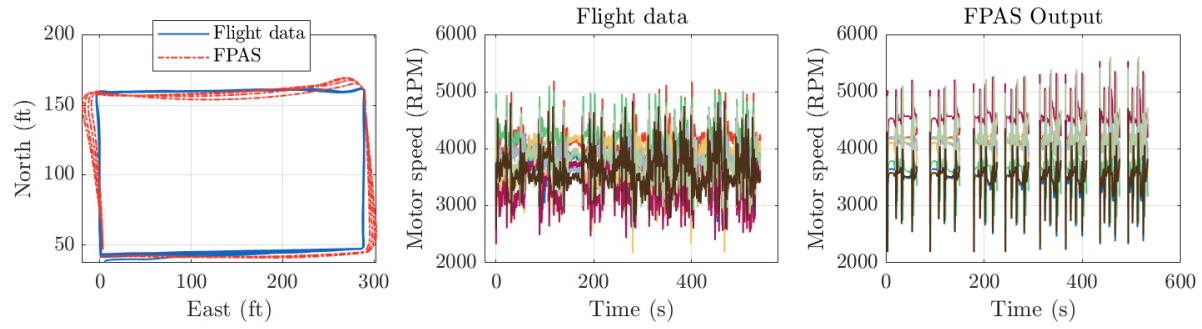
V. Example Flight Test Comparison

To evaluate FPAS’s ability to predict multirotor flight trajectories, a flight test campaign was conducted using GAMMA (Generic Aerodynamic Modeling for Multirotor Aircraft), a small octocopter developed at NASA LaRC in support of SWS goals [13, 15]. Photos of GAMMA on the ground and flying at the City Environment Range Testing for Autonomous Integrated Navigation (CERTAIN) at NASA LaRC are shown in Fig. 13. GAMMA is constructed from a commercial off-the-shelf Tarot X8 airframe and is equipped with eight KDE4213-XF brushless DC electric motors and Zubax Myxa B2 electronic speed controllers (ESCs) which each drive 15x5.5MR APC propellers. These propellers are the same model as those tested in a wind tunnel to build up the look-up tables for the simulation model [6]. The total weight of the vehicle is 18.6 lb, and a 20mAh 6S 12C Lithium-polymer battery was utilized to power the vehicle. A Pixhawk Cube Orange+ autopilot running PX4 open-source autopilot firmware is used to conduct autonomous flight tests and record data, and the open-source GCS software QGroundControl is used for mission planning and vehicle firmware configuration.

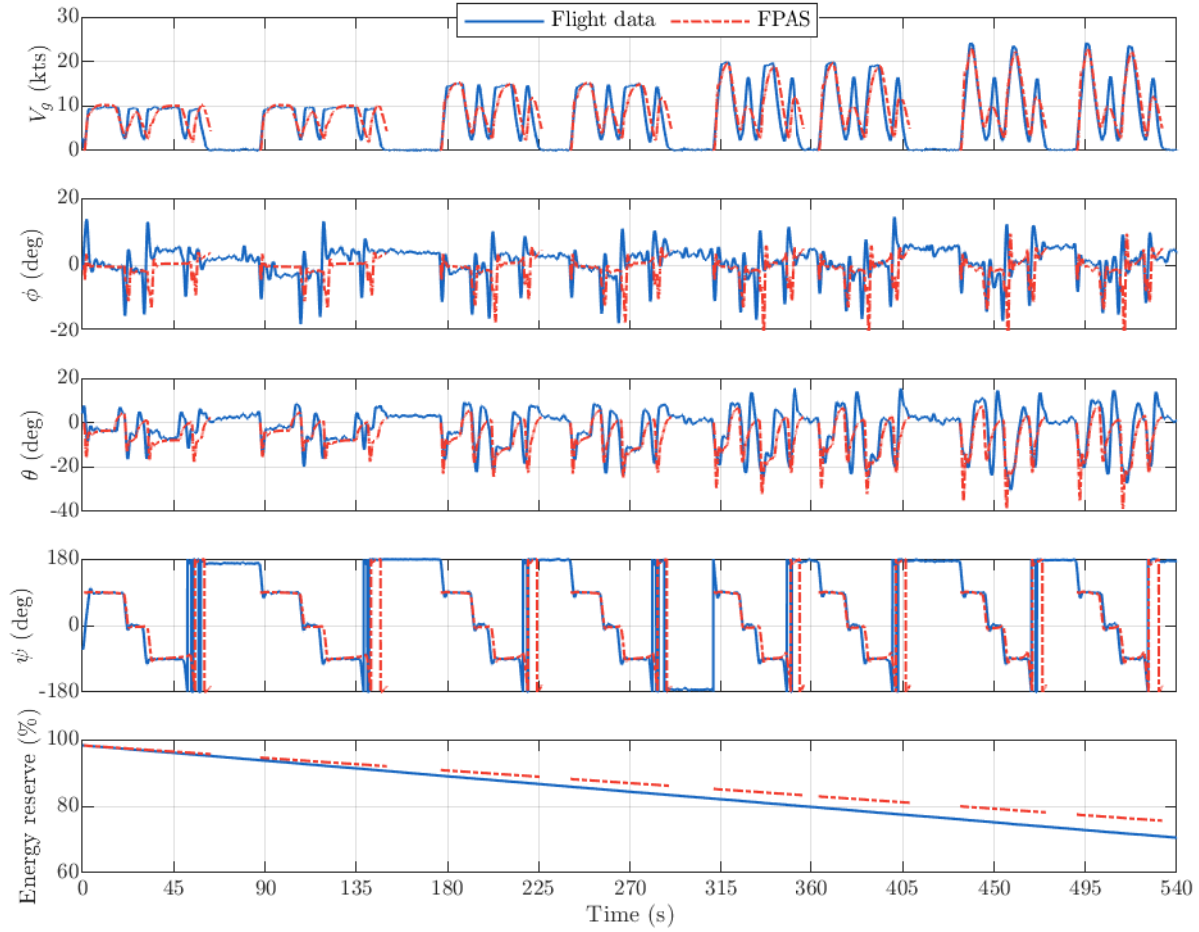
To collect data for comparison, a rectangular flight plan was flown at multiple different target ground speeds ranging from 10 to 25 kts. The flight was conducted using PX4’s “Mission” mode, which is an autonomous waypoint-following flight mode. At the time of flight, the wind was measured at approximately 4 kts from the west (280°) by a Vaisala WXT530 Series weather on the roof of a nearby trailer. The ground track of the flight is shown in Fig. 14a with the predicted trajectory from FPAS overlaid on the plot. The measured motor speeds and predicted motor speeds are also shown. In general, FPAS predicts the trajectory closely, although there is some error in the ground track due to differences in guidance algorithms. The motor speeds fall within a similar range over the course of the flight, suggesting that the underlying vehicle aerodynamics are modeled accurately. Vehicle velocity, attitude angles, and energy reserve are shown in Fig 14b. In these plots, there are gaps in the FPAS output to synchronize with the flight data because the



Fig. 13 GAMMA on the ground (left) and in flight at CERTAIN at NASA LaRC (right).



(a) Ground track and motor RPMs.



(b) Vehicle ground speed, attitude angles, and battery level.

Fig. 14 Comparison of FPAS outputs and flight test data.

vehicle stopped and hovered manually after completion of each lap before starting the next. The FPAS outputs match closely with each of the measured variables, although there is some difference in timing. FPAS also predicts a slightly larger spread between motor speeds than is present in the flight data. Energy reserve was computed from flight data by integrating the sum of motor power measurements, calculated as the product of current and voltage measured by the ESCs. This provides a one-to-one comparison with the FPAS output, which is computed the same way. Battery voltage, which is commonly monitored in-flight to predict remaining flight time, is not an FPAS output and cannot be compared directly. The energy reserve is predicted fairly accurately, with an error of just 4.5% at the end of the flight. This error could be caused simply by modeling error or the result of unmodeled phenomena such as turbulence or aerodynamic interactions. Accurate predictions of energy level are critical for pre-flight planning applications, where flight plan feasibility is assessed based on predicted battery margins.

In general, this example demonstrates the usefulness of FPAS as a trajectory prediction and performance assessment tool. Additional comparisons with vehicles that are dissimilar to those used to develop the simulation model would provide a measurement of FPAS's robustness to aircraft configuration. Furthermore, guidance algorithms will have a significant impact on the predicted flight path, as shown by this example. Updates to the trajectory algorithm to more closely align with the most popular open-source autopilots would improve the utility of FPAS.

VI. Conclusion

The development of a Flight Performance Assessment Service was detailed and presented. This service is designed to provide users with the capability to simulate and predict flight trajectories and performance characteristics from a predefined flight plan. These predictions could be used to assist pre-flight planning or in-flight rerouting. FPAS was developed as a standalone C++ application using code generated by the Mathworks Embedded Coder. A modular architecture was designed that facilitates the addition of new vehicle simulation models and communication with additional external services. An external commercial weather service can be queried to provide weather data for vehicle simulations. FPAS runs very quickly as a single executable file, allowing for batch processing that may be useful for pre-flight or in-flight applications.

FPAS employs a large database of aerodynamic and power consumption models developed from extensive wind tunnel testing of multirotor aircraft. These aerodynamic models are combined with a guidance and control architecture that mimics typical sUAS autopilots and enables the vehicle to follow a flight plan with specified waypoints. FPAS can provide predictions for a large variety of multirotor vehicles and flight conditions, including both nominal and failure conditions. The accuracy of the underlying aerodynamic models was demonstrated through a comparison between FPAS predictions and flight test data of a small multirotor aircraft. The predicted outputs, including the flight trajectory and energy consumption and reserve, matched the flight data within an acceptable margin to be useful for typical use-cases such as trajectory planning and performance assessment.

Future directions for this work include incorporating FPAS into a real-time IASMS framework by communicating with other SFCs developed as part of the SWS project, such in-flight trajectory optimization services or as battery monitoring services that could be used to estimate remaining battery voltage based on FPAS outputs. The simulation models are continually being developed to improve realism in areas such as electric motor dynamics and guidance algorithms. Additionally, flight testing is underway to compare FPAS with more vehicles, improve the quality and breadth of the aerodynamic models, and include complex phenomena such as rotor interaction effects, which can present a flight hazard and cause a significant increases in power consumption at particular flight conditions. Finally, work is underway to develop a routine to automatically generate low-fidelity models from user inputs that can then be used to simulate and predict flight paths even more rapidly in exchange for decreased model fidelity.

Acknowledgments

This research was funded by the NASA System-Wide Safety project.

References

- [1] "NASA Aeronautics Strategic Implementation Plan 2023," <https://www.nasa.gov/directorates/armd/armd-strategic-implementation-plan/>, 2023. Accessed 23 May, 2025.
- [2] National Academies of Sciences, Engineering, and Medicine, "In-Time Aviation Safety Management: Challenges and Research for an Evolving Aviation System," Washington, DC: The National Academies Press, 2018. <https://doi.org/https://doi.org/10.17232/navsafety>

//doi.org/10.17226/24962.

- [3] International Civil Aviation Organization (ICAO), "Safety Management," ICAO Annex 19 to the Convention on International Civil Aviation Standards and Recommended Practices, 2016.
- [4] Moore, A. J. et al., "Testing of Advanced Capabilities to Enable In-time Safety Management and Assurance for Future Flight Operations," NASA TM-20230018665, April 2024.
- [5] Young, Steven. D. et al., "Architecture and Information Requirements to Assess and Predict Flight Safety Risks During Highly Autonomous Urban Flight Operations," NASA TM-20200001140, January 2020.
- [6] Foster, J. V., and Hartman, D., "High-Fidelity Multi-Rotor Unmanned Aircraft System (UAS) Simulation Development for Trajectory Prediction Under Off-Nominal Flight Dynamics," *17th AIAA Aviation Technology, Integration, and Operations Conference*, 2017. <https://doi.org/10.2514/6.2017-3271>.
- [7] Hartman, D., "Identification of Hazardous Flight Conditions to Establish a Safe Flight Envelope for Autonomous Multirotor Aircraft," *AIAA Scitech 2019 Forum*, 2019. <https://doi.org/10.2514/6.2019-1292>.
- [8] Foster, J. V., Miller, L. J., Busan, R. C., Langston, S., and Hartman, D., "Recent NASA Wind Tunnel Free-Flight Testing Of A Multirotor Unmanned Aircraft System," *AIAA Scitech 2020 Forum*, 2020. <https://doi.org/10.2514/6.2020-1504>.
- [9] Altamirano, G. V., Matt, J. J., Busan, R. C., and Foster, J. V., "Wind Tunnel Testing of Static Aerodynamic and Power Consumption Characteristics of an Octocopter," *AIAA AVIATION 2023 Forum*, 2023. <https://doi.org/10.2514/6.2023-3967>.
- [10] Altamirano, G. V., Matt, J. J., Farrell, S., , and Palazzo, A., "Static and Forced Oscillation Wind Tunnel Tests of a Tandem Propeller Configuration," *AIAA SciTech 2025 Forum*, 2025.
- [11] Young, S. D., Ancel, E., Dill, E. T., Moore, A., Quach, C. C., Smalling, K. M., and Ellis, K. K., "Flight Testing In-Time Safety Assurance Technologies for UAS Operations," *AIAA AVIATION 2022 Forum*, 2022. <https://doi.org/10.2514/6.2022-3458>.
- [12] Stevens, B. L., Lewis, F. L., and Johnson, E. N., *Aircraft Control and Simulation*, 3rd ed., Wiley, Hoboken, NJ, 2016.
- [13] Matt, J. J., and Altamirano, G. V., "System Identification of an Octocopter in Hover using Full-Harmonic Orthogonal Multisine Inputs," *AIAA AVIATION 2025 Forum*, 2025.
- [14] Leslie, R., Geyer, D., Cunningham, K., Glaab, P., Kenney, P., and Madden, M., "LaSRS++ - An object-oriented framework for real-time simulation of aircraft," *AIAA Modeling and Simulation Technologies Conference and Exhibit*, 1998. <https://doi.org/10.2514/6.1998-4529>.
- [15] Corrigan, P. E., Matt, J. J., and Asper, G. D., "Design and Testing of an Octocopter for Aerodynamic and Power Consumption Modeling," NASA TM-20240013453, to appear 2025.